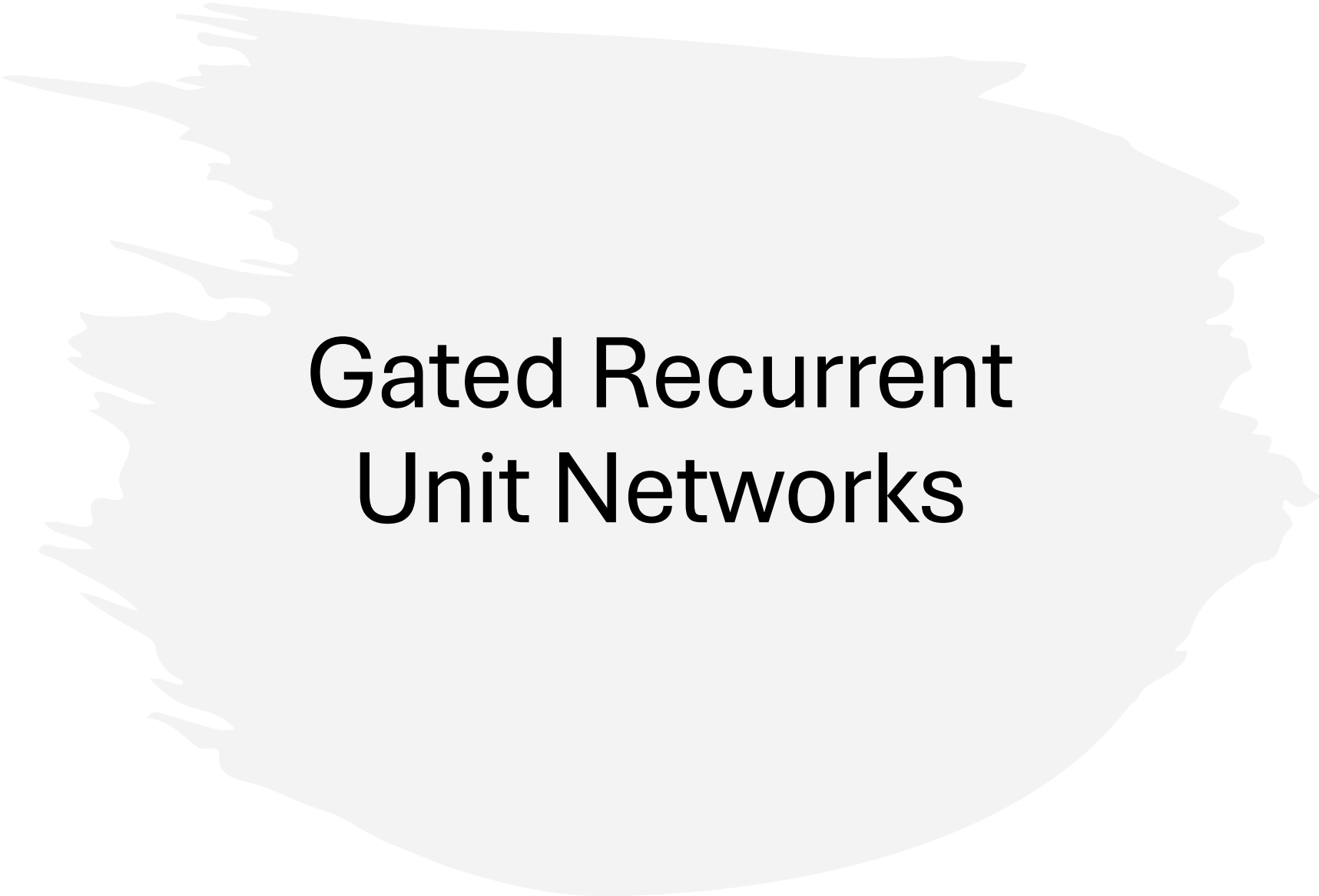




Gated Recurrent Unit Networks



Assignment 1

Implement a simple GRU cell from scratch using NumPy. Your implementation should define the update gate, reset gate, and candidate activation, then compute the final hidden state. Use a random input vector and initial hidden state for testing.

Step 1

- Define the GRU cell equations:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



Step 2: Initialize parameters

- Define weight matrices and biases.
 - Initialize a test input and previous hidden state.
 - Implement the forward pass using NumPy
-

Step 3: Define GRU Cell function

```
FUNCTION GRU_Cell(x_t, h_t_minus_1, W_z, W_r, W_h, U_z, U_r, U_h, b_z, b_r, b_h):
```

```
    # Compute Update Gate
```

```
    z_t = Sigmoid(W_z * x_t + U_z * h_t_minus_1 + b_z)
```

```
    # Compute Reset Gate
```

```
    r_t = Sigmoid(W_r * x_t + U_r * h_t_minus_1 + b_r)
```

```
    # Compute Candidate Hidden State
```

```
    h_tilde = tanh(W_h * x_t + U_h * (r_t  $\odot$  h_t_minus_1) + b_h)
```

```
    # Compute Final Hidden State
```

```
    h_t = (1 - z_t)  $\odot$  h_t_minus_1 + z_t  $\odot$  h_tilde
```

```
    RETURN h_t
```

```
END FUNCTION
```

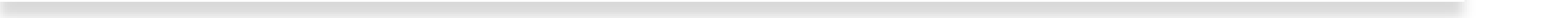
Assignment 2

- Train a ***Gated Recurrent Unit (GRU)*** model for sentiment analysis using the IMDb movie reviews dataset.
- Dataset: **IMDb Large Movie Review Dataset**
- Access the dataset here:
 - <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

Step 1: Data Preprocessing



Load the dataset and preprocess the text data by:

- Tokenizing the reviews.
 - Removing stop words and punctuation.
 - Padding sequences to ensure uniform input length.
- 

Preprocessing function

Function **LoadAndPreprocessTextData**(dataset_path, vocab_size, max_length):

Step 1: reviews_list $\leftarrow$ Load_Dataset(dataset_path)

Step 2: tokenized_sequences $\leftarrow$ Tokenize_Text(reviews_list, vocab_size)

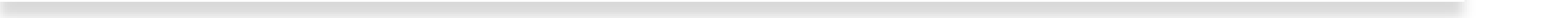
Step 3: cleaned_sequences $\leftarrow$ Remove_Stopwords_And_Punctuation (tokenized_sequences, stopwords_list, punctuation_list)

Step 4: preprocessed_sequences $\leftarrow$ Pad_Sequences(cleaned_sequences, max_length)

Return preprocessed_sequences

Step 2: Data Splitting



- Divide the dataset into training and testing sets, ensuring an 80-20 split.
 - Highlight the importance of training and testing splits in evaluating model performance.
- 

Step 3: Model Architecture Design

- Design a GRU-based neural network with:
 - An embedding layer to convert words into vector representations.
 - A GRU layer to capture sequential dependencies.
 - A dense layer with a sigmoid activation function for binary classification.

Step 4

- Compile the model using an appropriate loss function and optimizer.
- Train the model on the training data, monitoring performance on the validation set.
- Evaluate the model's performance on the test set using metrics like accuracy, precision, recall, and F1-score.



Thank you

